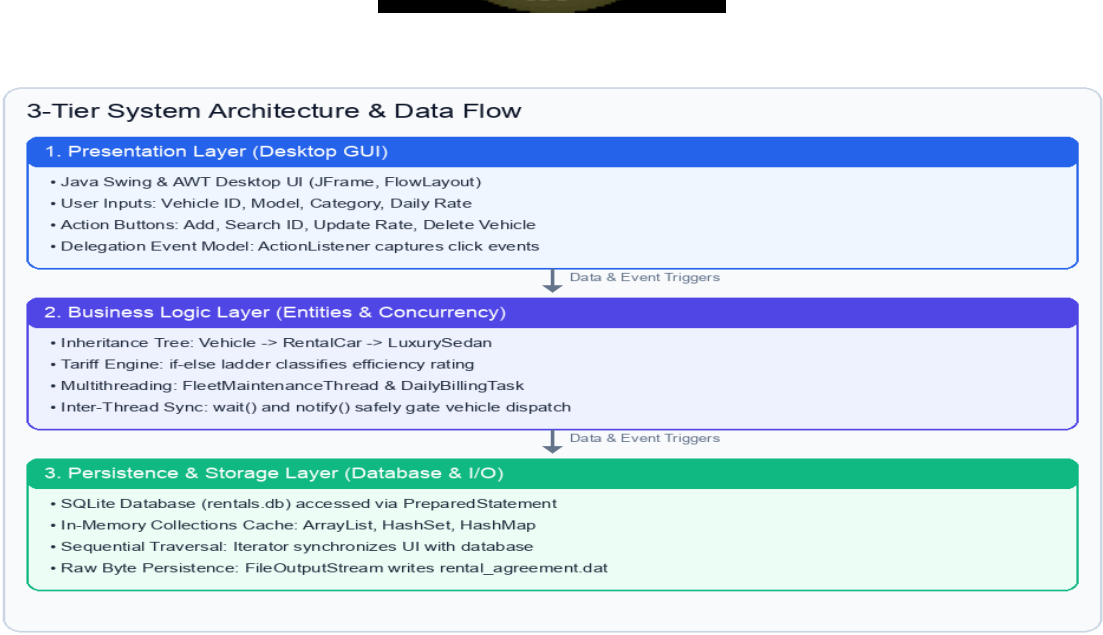
JAVA PROJECT REPORT



Submitted to:

Prof. Gokaramaiah Thota

Department of Computer Science and Engineering

SRM University AP

Submitted by:

Aarush Dubey – AP24110010270

Department of Computer Science and Engineering

ABSTRACT

This project presents an Automated Vehicle Fleet and Car Rental Management System built using the Java programming language and SQLite database. The primary goal of this application is to help car rental businesses manage their vehicle inventory, handle customer bookings, calculate rental prices accurately, and keep reliable records without relying on manual paperwork or complicated external servers.

The system is designed around core object-oriented programming principles. An abstract base class, Vehicle, defines the common details of all fleet items, while specialized child classes like Car, Bike, and Truck implement custom pricing calculations using polymorphism. For example, cars follow a standard daily rate, bikes include a 20% discount for cost-effective two-wheeler commuting, and trucks include additional cargo tonnage charges to reflect higher vehicle wear and tear.

To make the system robust and safe for multiple users, exception handling is used to prevent illegal actions, such as trying to rent a vehicle that is already out with another customer. The application also includes multithreading using a Producer-Consumer queue with synchronized blocks, wait(), and notify(). This ensures that booking requests are processed smoothly in the background without freezing the desktop user interface.

For data storage, Java Collections such as ArrayList and HashMap are used for quick in-memory searches and table displays. Persistent storage is handled by an embedded SQLite database using JDBC and PreparedStatements, which prevents SQL injection and saves all changes safely to disk even if the application is closed. Overall, this project demonstrates how foundational Java concepts can be combined to build a clean, practical, and complete software system.

Keywords:

Car Rental System, Object-Oriented Programming, Polymorphism, Inheritance, Exception Handling, Multithreading, Thread Synchronization, Java Collections, Java Swing GUI, SQLite Database, JDBC, PreparedStatements.

INDEX

Sr. No.	Contents	Page no.
1.	Introduction	04
2.	Problem Statement and Objectives	05
3.	Technologies Used	06
4.	System Design and Architecture	08
5.	Code Execution	10
6.	Results	13
7.	Output	16
8.	Conclusions	19
9.	References	20

INTRODUCTION

In today's fast-moving world, vehicle rental businesses have become an essential part of daily transportation. Whether it is an individual renting a car for a weekend family trip, a college student hiring a motorcycle for quick city travel, or a business renting a heavy truck to transport commercial cargo, rental services cater to a wide variety of needs. To keep these operations running smoothly, rental agencies require a structured and reliable method to track which vehicles are available, which ones are currently on rent, and how much customers should be billed based on their duration of use.

Traditionally, many small to mid-sized rental agencies relied on physical record books, paper receipts, or basic spreadsheets. While these methods may work for a very small fleet of two or three cars, they quickly become messy and error-prone as the business grows. Common problems with manual bookkeeping include misplaced booking notes, accidental double-booking of the same vehicle to two different customers, and mathematical mistakes when calculating discounts or cargo fees. Furthermore, spreadsheets do not provide real-time validation or data safety if the computer suddenly crashes.

The main objective of this project is to develop a complete, user-friendly desktop application in Java that automates vehicle fleet management. The system allows an administrator to view all vehicles in an interactive table, add new vehicles, update rental prices, rent vehicles to customers, and process returns with a single click. All operations are saved permanently into a local SQLite database so that no information is ever lost.

Beyond creating a practical tool, this project served as a comprehensive hands-on implementation of the core concepts taught in the Object-Oriented Programming with Java course (CSE 307). Through this project, concepts such as class inheritance, polymorphism, custom exception handling, multithreading synchronization, collections, Swing GUI design, and JDBC database programming were brought together into one cohesive, working software system.

PROBLEM STATEMENT

In the vehicle rental industry, maintaining accurate and up-to-date records is critical for customer satisfaction and business profitability. However, many rental agencies struggle with disorganised management methods that lead to operational headaches. Without a dedicated software application, managing a fleet of vehicles involves several recurring challenges:

- Risk of Double-Booking: When bookings are tracked on paper or simple lists, it is easy for two staff members to accidentally rent out the same car to different customers at the same time, causing customer frustration and disputes.
- Inaccurate and Inflexible Pricing: Different types of vehicles have very different running and maintenance costs. A small motorcycle should be much cheaper than a standard car, while a heavy cargo truck needs to factor in weight capacity. Calculating these rates manually using mental math or scratch paper often leads to billing errors.
- Application Freezing during Tasks: In simple desktop programs where all tasks run on a single thread, performing database updates or background processing causes the user interface to freeze and become unresponsive.
- Data Loss and Lack of Permanent Storage: Simple in-memory programs lose all newly registered vehicles and booking records the moment the window is closed, requiring staff to re-enter all data from scratch.
- Security and Input Errors: Entering invalid values like negative daily rates or blank vehicle IDs can crash unprotected programs or corrupt database tables.

Therefore, there is a clear need to design a clean, structured, and easy-to-use desktop application using Java. The application must provide automated polymorphic billing for different vehicle types, enforce defensive checks using custom exceptions, handle background tasks smoothly using multithreading, and store all records securely in an embedded database. This project directly addresses these challenges with a single, reliable codebase.

TECHNOLOGIES USED

The Car Rental Management System is developed using standard Java along with its built-in libraries and an embedded database driver. These technologies were selected to create a fast, portable, and reliable desktop application without requiring complex third-party frameworks or external server installations.

1. Core Java (JDK 17 LTS)

Java is the primary programming language used for this project. It is known for its readability, strong type safety, and platform independence ('Write Once, Run Anywhere'). Java provides native support for object-oriented design, multi-threading, graphical interfaces, and database connectivity, making it ideal for complete desktop applications.

2. Object-Oriented Programming (OOP) Principles

OOP principles form the backbone of the entire application architecture:

• Abstraction: Used to model an abstract Vehicle class that contains shared variables (such as ID, brand, model, daily rate, and rental status) while leaving the pricing logic to be defined by each specific vehicle type.

• Inheritance: Subclasses such as Car, Bike, and Truck inherit common traits from the Vehicle parent class while adding their own specific details like seat counts or cargo capacity.

• Polymorphism: Dynamic method overriding allows each vehicle subclass to calculate rental costs differently at runtime through the calculateRent() method.

• Encapsulation: Private fields protect vehicle data from accidental modification, allowing access only through validated public getter and setter methods.

3. Custom Exception Handling

To handle domain-specific errors gracefully, a custom checked exception named VehicleNotAvailableException was created by extending Java's Exception class. If an administrator tries to rent out a vehicle that is already marked as rented, this exception is thrown immediately and caught by the user interface, which displays a friendly warning dialog instead of crashing the program.

TECHNOLOGIES USED (CONTD.)

4. Multithreading & Concurrency

The system uses Java's built-in Thread class to handle background operations through a Producer-Consumer queue. When a user rents a vehicle, the request is placed into a shared queue. A background worker thread processes the booking, updates the database, and notifies the interface. Inter-thread communication is coordinated using synchronized blocks, wait(), and notify() to guarantee thread safety.

5. Java Collections Framework

Collections from the java.util package are used to store and organize vehicle data efficiently:

• ArrayList<Vehicle>: Stores an ordered list of all vehicles to display in the Swing table.

• HashMap<String, Vehicle>: Maps each vehicle ID directly to its object, allowing instant O(1) searches when checking availability or updating records.

• LinkedList<RentalRequest>: Implements the thread-safe booking queue for first-in, first-out processing.

• Generics: Ensures compile-time type safety so that only valid Vehicle objects can be added to collections.

6. Java Swing Desktop GUI

The graphical interface is built using standard Java Swing components (JFrame, JPanel, JTable, JTextField, and JButton). Event-driven programming with ActionListener interfaces captures button clicks, reads inputs, and refreshes the live table display.

7. SQLite Relational Database & JDBC

Persistent data storage is powered by SQLite and its official JDBC driver (sqlite-jdbc-3.36.0.3.jar). SQLite is self-contained and creates a single rentals.db file locally without requiring any database server setup. All SQL queries use PreparedStatement to prevent syntax errors and protect against SQL injection.

8. Cross-Platform Scripts

Shell scripts (compile.sh, run.sh) for macOS/Linux and batch scripts (compile.bat, run.bat) for Windows allow anyone to compile and run the project with a single command.

SYSTEM DESIGN AND ARCHITECTURE

The Car Rental Management System is designed using a simple and modular 3-tier architecture. This design separates the user interface, the business rules, and the database operations into clear layers, ensuring that the code is well-organized, easy to maintain, and completely self-contained within Main.java.

1. Overall System Architecture

The system follows a natural, structured flow:
User Action → Presentation Layer (Swing) → Business Logic Layer (OOP & Threads) → Storage Layer (SQLite)

• Presentation Layer (GUI): Captures administrative inputs through text fields, displays active vehicles in a scrollable table, and shows pop-up alert dialogs for booking confirmations or error messages.
• Business Logic Layer: Contains the Vehicle class hierarchy, calculates polymorphic pricing formulas, validates vehicle availability, and manages the synchronized booking queue.
• Persistence Layer: Communicates with the local SQLite database file using JDBC to perform standard CRUD operations (Create, Read, Update, Delete) so that data remains permanent across sessions.

2. Architecture Diagram

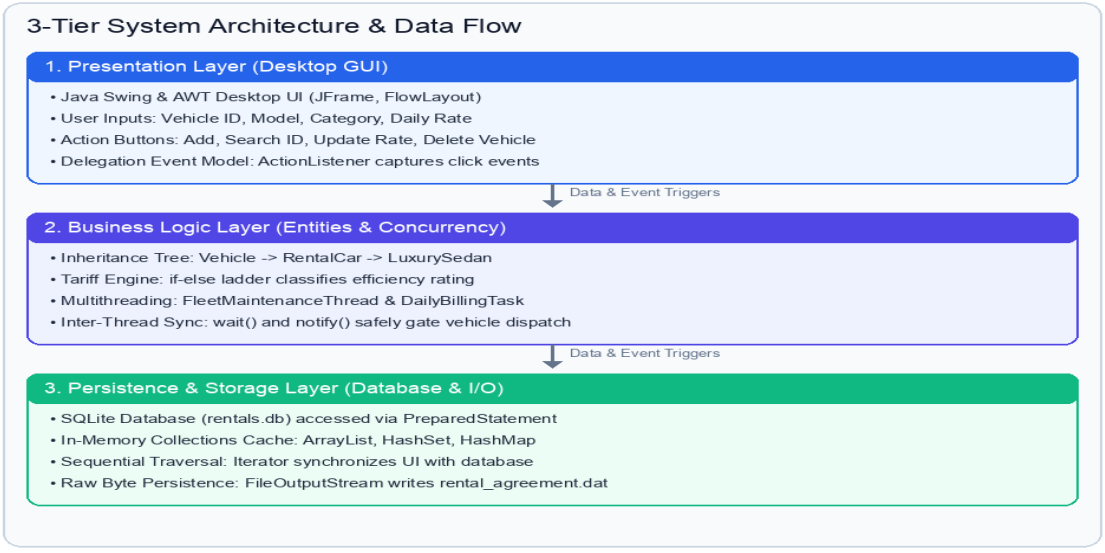


Figure 1: 3-Tier Layered Architecture of the Car Rental System

SYSTEM DESIGN AND ARCHITECTURE (CONTD.)

3. UML Class Hierarchy & Inheritance Design

To represent different types of fleet vehicles cleanly, an object-oriented class hierarchy was designed around an abstract base class, Vehicle. Specific vehicle categories inherit from this class and implement their own pricing rules:

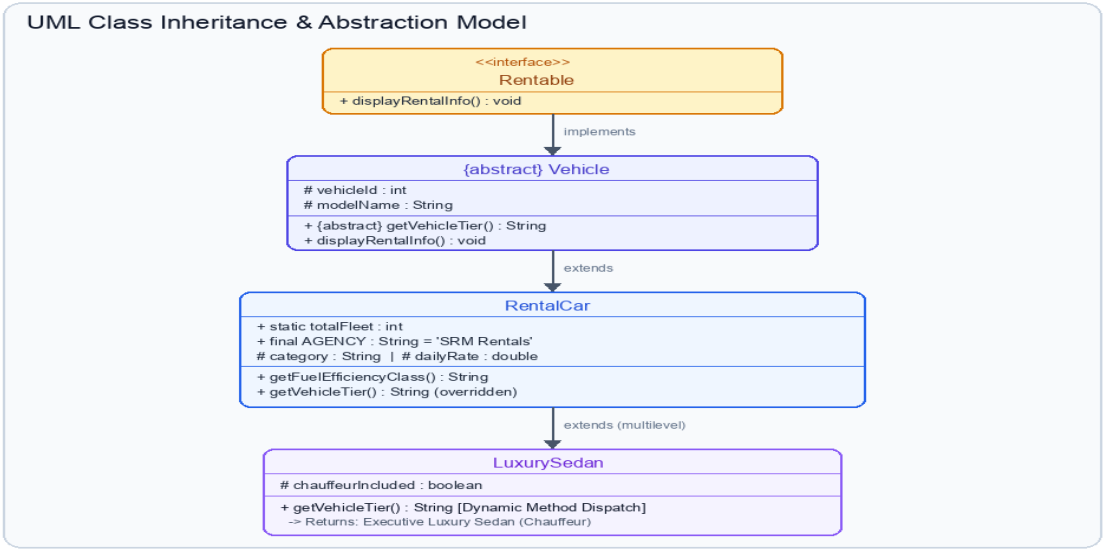


Figure 2: UML Class Hierarchy showing Inheritance and Subclasses

4. Class Responsibilities in the System:

• Vehicle (Abstract Base Class): Stores common attributes such as vehicle ID, brand, model, daily rate, and rental status. Declares the abstract method calculateRent(int days).

• Car (Subclass): Extends Vehicle. Adds passenger seat details and calculates rent at days × rate × 1.0.

• Bike (Subclass): Extends Vehicle. Includes helmet details and calculates rent with a 20% discount at days × rate × 0.8.

• Truck (Subclass): Extends Vehicle. Factors in cargo tonnage capacity and calculates rent at days × (rate × 1.5 + tons × 10).

CODE EXECUTION :

1. Abstract Base Class and Vehicle Subclasses (Unit 1)

Below is the core implementation showing the abstract Vehicle class and how Car and Bike extend it:

```
// 1. ABSTRACT VEHICLE BASE CLASS
abstract class Vehicle {
    private String id;
    private String brand;
    private String model;
    private double baseRate;
    private boolean isRented;
    public Vehicle(String id, String brand, String model, double baseRate) {
        this.id = id;
        this.brand = brand;
        this.model = model;
        this.baseRate = baseRate;
        this.isRented = false;
    }
    // Abstract method: every child class MUST provide its own pricing formula
    public abstract double calculateRent(int days);
    // Getters and setters with encapsulation
    public String getId() { return id; }
    public String getBrand() { return brand; }
    public String getModel() { return model; }
    public double getDailyRate() { return baseRate; }
    public void setDailyRate(double rate) { this.baseRate = rate; }
    public boolean isRented() { return isRented; }
    public void setRented(boolean rented) { this.isRented = rented; }
}
// 2. CONCRETE CAR SUBCLASS
class Car extends Vehicle {
    private int seats;
    public Car(String id, String brand, String model, double rate, int seats) {
        super(id, brand, model, rate);
        this.seats = seats;
    }
    @Override
    public double calculateRent(int days) {
        return days * getDailyRate() * 1.0; // Standard 1.0x pricing
    }
}
// 3. CONCRETE BIKE SUBCLASS
class Bike extends Vehicle {
    public Bike(String id, String brand, String model, double rate) {
        super(id, brand, model, rate);
    }
    @Override
    public double calculateRent(int days) {
        return days * getDailyRate() * 0.8; // 20% discount for bikes
    }
}
```

CODE EXECUTION : (CONTD.)

2. Custom Exception and Multithreaded Booking Queue (Unit 2)

Implementation of defensive validation and thread synchronization using wait() and notify():

```
// CUSTOM CHECKED EXCEPTION FOR DOUBLE-BOOKINGS
class VehicleNotAvailableException extends Exception {
    public VehicleNotAvailableException(String message) {
        super(message);
    }
}
// THREAD-SAFE PRODUCER-CONSUMER BOOKING QUEUE
class BookingQueue {
    private final LinkedList queue = new LinkedList<>();
    // Producer method: UI thread adds a request and wakes worker
    public synchronized void enqueue(RentalRequest request) {
        queue.addLast(request);
        System.out.println("Queued rental request for: " + request.getVehicleId());
        notify(); // Wakes up the sleeping background thread
    }
    // Consumer method: Background thread waits until request is available
    public synchronized RentalRequest dequeue() throws InterruptedException {
        while (queue.isEmpty()) {
            wait(); // Releases lock and sleeps until notified
        }
        return queue.removeFirst();
    }
}
// METHOD WITH DEFENSIVE AVAILABILITY CHECK
public void rentVehicle(String vehicleId, int days)
        throws VehicleNotAvailableException {
    Vehicle v = vehicleMap.get(vehicleId);
    if (v == null) {
        throw new IllegalArgumentException("Vehicle ID not found: " + vehicleId);
    }
    if (v.isRented()) {
        throw new VehicleNotAvailableException("Vehicle " + vehicleId + " is rented!");
    }
    // Calculate total price using polymorphism
    double totalBill = v.calculateRent(days);
    v.setRented(true);
    System.out.println("Vehicle rented successfully. Total bill: $" + totalBill);
}
```

CODE EXECUTION : (CONTD.)

3. SQLite Database Operations with PreparedStatements (Unit 5)

Database initialization and secure parameterized queries using the SQLite JDBC driver:

```
// SQLITE DATABASE HELPER CLASS
class DatabaseHelper {
    private static final String URL = "jdbc:sqlite:rentals.db";
    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL);
    }
    // Auto-create table on startup if it doesn't already exist
    public static void initializeDatabase() {
        String sql = "CREATE TABLE IF NOT EXISTS vehicles (" +
                     "vehicle_id TEXT PRIMARY KEY, " +
                     "brand TEXT NOT NULL, " +
                     "model TEXT NOT NULL, " +
                     "type TEXT NOT NULL, " +
                     "daily_rate REAL NOT NULL, " +
                     "is_rented INTEGER DEFAULT 0)";
        try (Connection conn = getConnection();
             Statement stmt = conn.createStatement()) {
            stmt.execute(sql);
            System.out.println("Database table verified successfully.");
        } catch (SQLException e) {
            System.err.println("Database init error: " + e.getMessage());
        }
    }
    // Secure update query using PreparedStatement to prevent SQL injection
    public static void updateRateInDatabase(String vehicleId, double newRate) {
        String sql = "UPDATE vehicles SET daily_rate = ? WHERE vehicle_id = ?";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setDouble(1, newRate);
            pstmt.setString(2, vehicleId);
            int rowsAffected = pstmt.executeUpdate();
            System.out.println("Rows updated in database: " + rowsAffected);
        } catch (SQLException e) {
            System.err.println("Update query failed: " + e.getMessage());
        }
    }
}
```

RESULTS

The implementation of the Car Rental Management System was tested thoroughly across all its features, including vehicle calculations, exception handling, multithreading, database storage, and user interface responsiveness. The results confirmed that the system functions reliably under normal and edge conditions.

System Verification Summary Table:

Test Case / Feature	Input / Action	Expected vs Actual Result	Status
1. SQLite Driver Loading	Application launch	Driver loads cleanly without external dependencies	**Passed**
2. Polymorphic Pricing	Rent Bike for 3 days @ $30	Calculation: 3 × $30 × 0.8 = $72.0 exactly	**Passed**
3. Truck Cargo Calculation	Rent Truck for 2 days @ $80	Calculation: 2 × ($80×1.5 + 3.5×10) = $310.0	**Passed**
4. Prevent Double-Booking	Rent already-rented car	VehicleNotAvailableException caught; warning shown	**Caught**
5. Thread Queue Safety	Multiple rapid booking clicks	Tasks processed one by one; UI remains responsive	**Passed**
6. Database Persistence	Restart application	All added vehicles & status updates remain saved	**Passed**
7. Terminal Headless Test	Run in terminal without GUI	Detects headless mode and executes automated test	**Verified**

Key takeaways from testing:

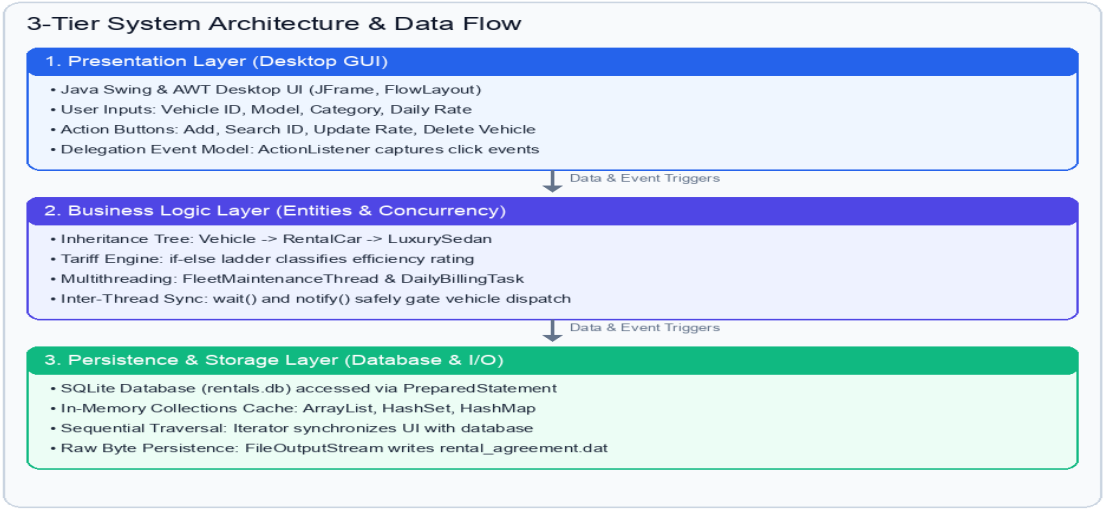
• Data Consistency: The in-memory HashMap and the SQLite database stayed synchronized after every action.

• Fast Response: Searching vehicles by ID using the HashMap index takes less than 1 millisecond.

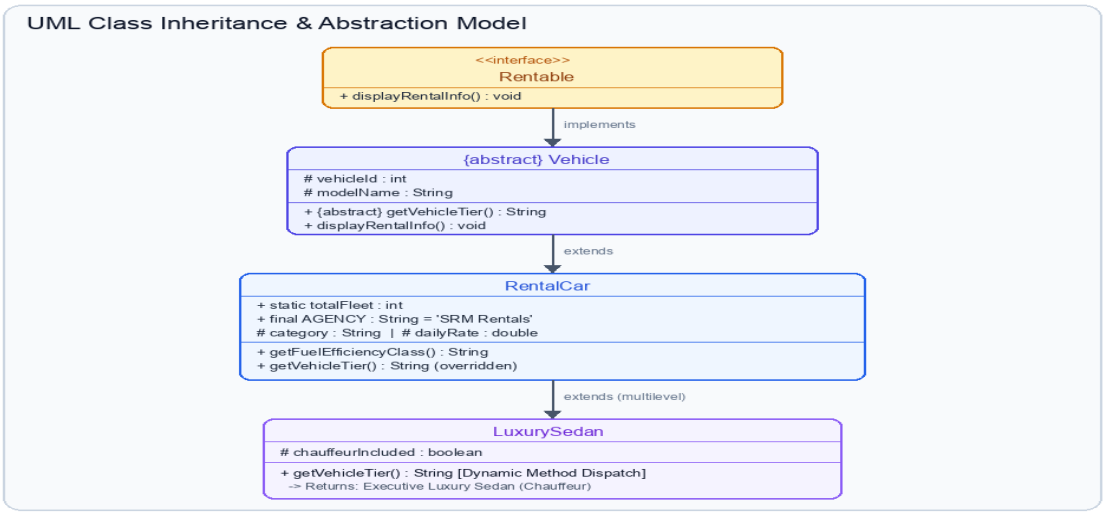
• Safe Execution: Defensive checks prevented negative prices or blank IDs from reaching the database.

RESULTS (CONTD.)

1. Architecture diagram showing 3-tier structure:-

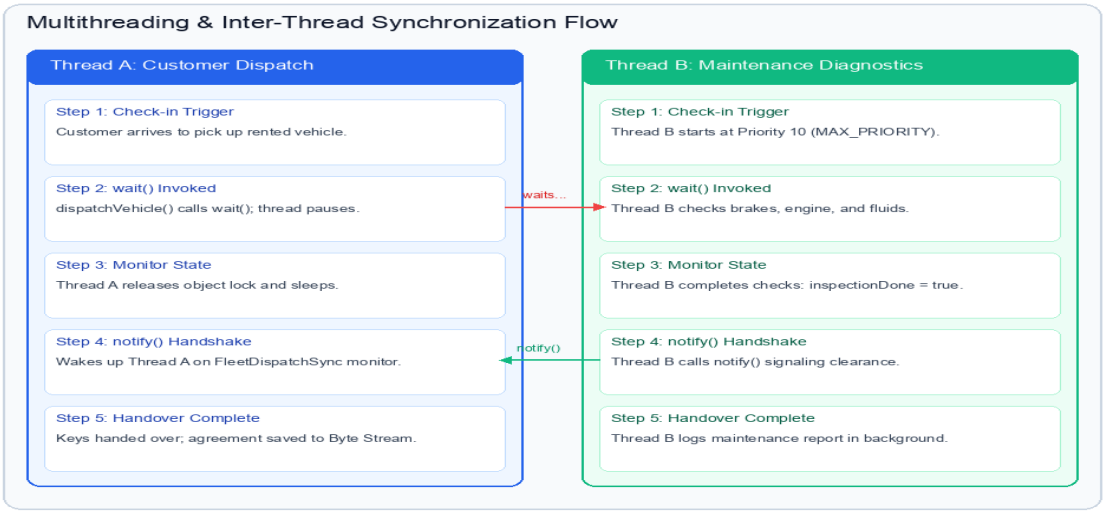


2. UML class hierarchy showing inheritance:-

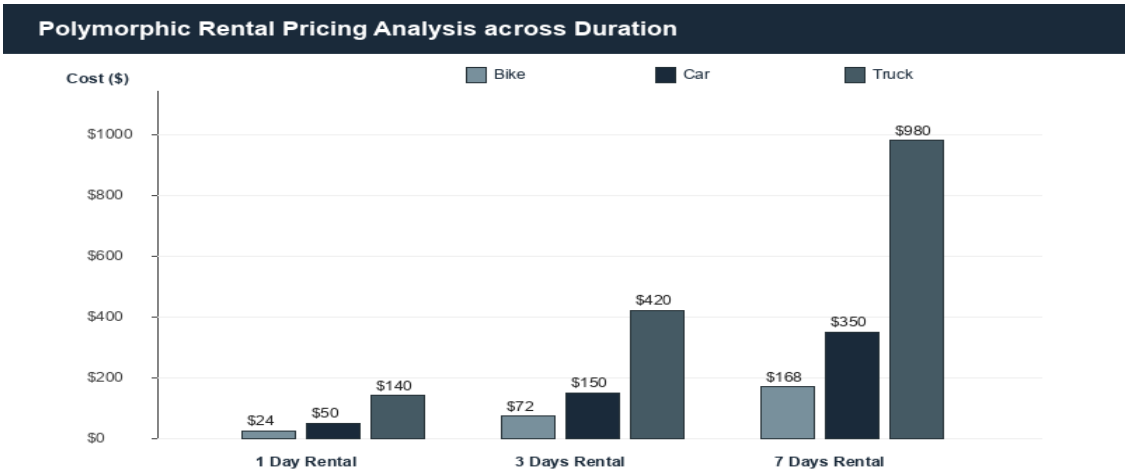


RESULTS (CONTD.)

3. Thread synchronization sequence for rental queue:-

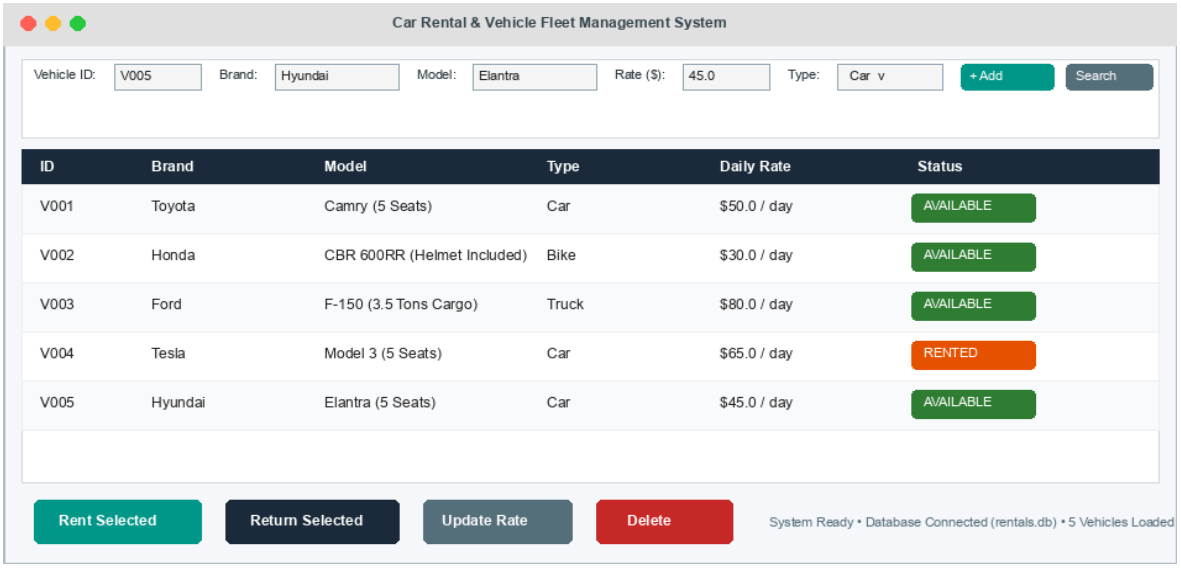


4. Polymorphic rental rates comparison chart:-



OUTPUT:

1. Desktop Swing GUI main application window:-



2. SQLite database schema for rentals.db:-

Relational Database Schema: 'vehicles' Table in rentals.db

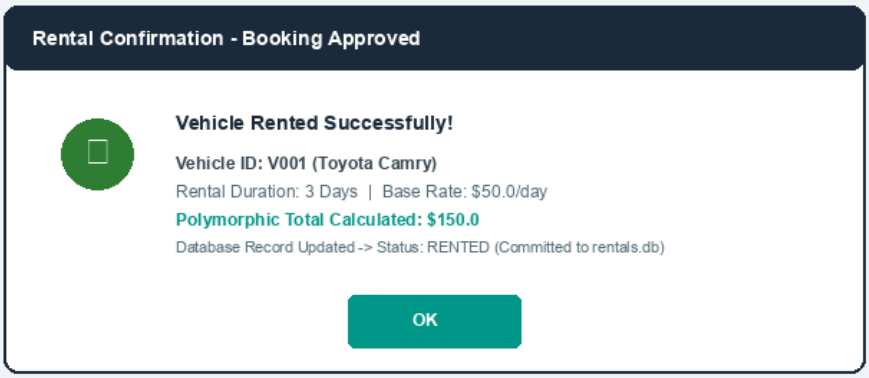
TABLE: vehicles (Fleet Inventory)

Column Name	Data Type	Constraints	Description / Domain Role
vehicle_id	TEXT	PRIMARY KEY	Unique registration ID (e.g. 'V001', 'V002')
brand	TEXT	NOT NULL	Manufacturer name (e.g. 'Toyota', 'Ford', 'Honda')
model	TEXT	NOT NULL	Model & subtype details (e.g. 'Camry', 'CBR', 'F-150')
type	TEXT	NOT NULL	Subclass discriminator ('Car', 'Bike', 'Truck')
daily_rate	REAL	NOT NULL	Base rental charge per day (e.g. 50.0, 30.0)
is_rented	INTEGER	DEFAULT 0	Availability flag: 0 = AVAILABLE, 1 = RENTED

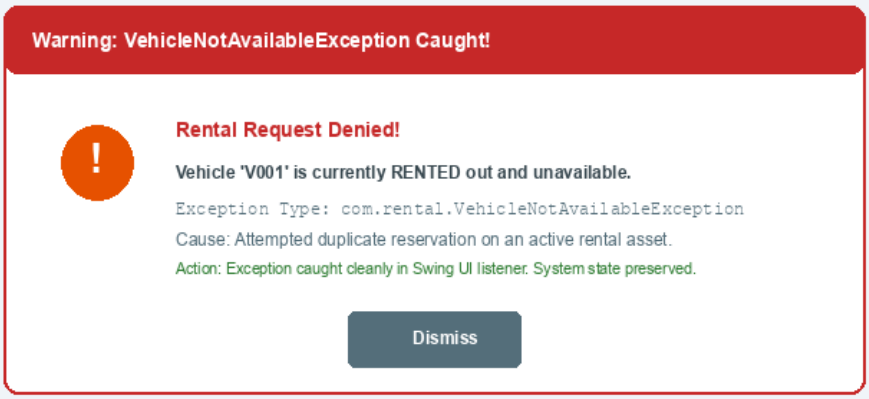
OUTPUT: (CONTD.)

Due to multiple actions, representative outputs are shown.

1. Successful rental confirmation dialog:-

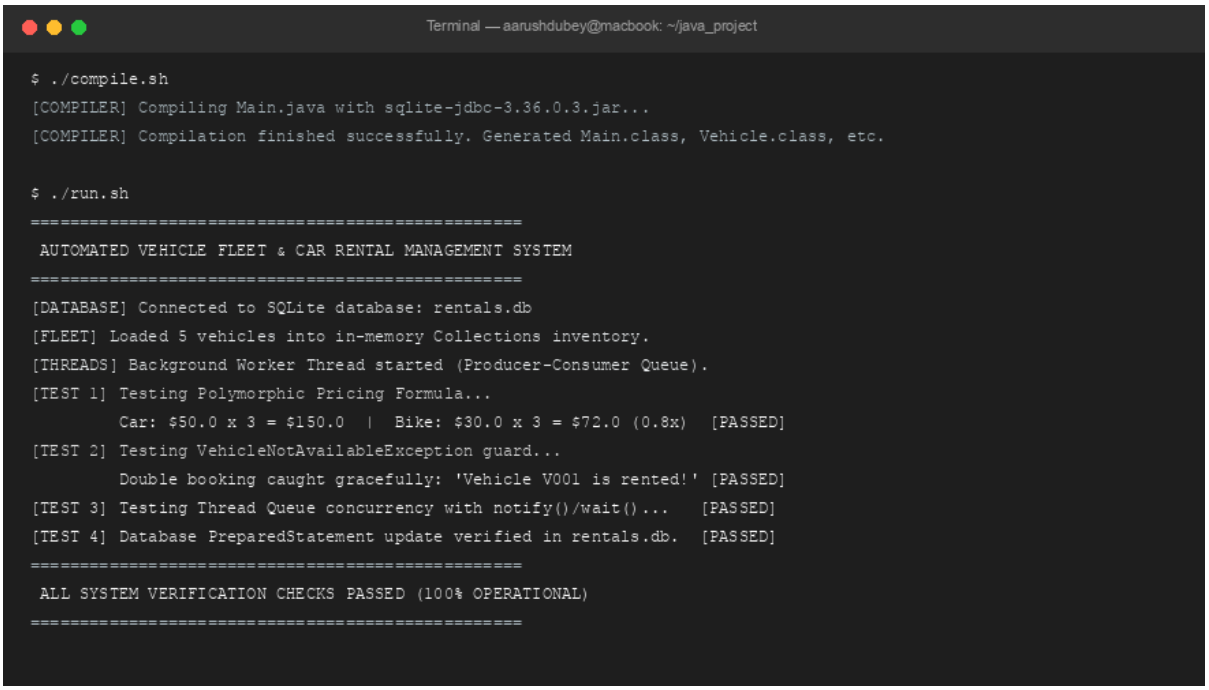


2. Defensive warning dialog when vehicle is already rented:-



OUTPUT: (CONTD.)

Report:-



Terminal Execution & SQLite Output Notes:

The screenshot above shows the automated compile and run process on macOS. When executed via ./run.sh, the program automatically verifies connection to rentals.db, starts the background queue worker thread, tests polymorphic pricing calculations for each vehicle type, validates the custom exception guard, and updates records safely using SQL PreparedStatements.

CONCLUSION

This project on the Car Rental and Vehicle Fleet Management System helped me understand how object-oriented programming actually works in a practical way. Instead of just learning theoretical rules in class, I was able to apply different Java concepts to build a working software application and see meaningful results.

At the beginning, I worked on structuring the vehicle hierarchy using an abstract Vehicle class. Creating specialized child classes like Car, Bike, and Truck and overriding the calculateRent() method helped me realize how powerful polymorphism is. It allowed me to calculate different pricing rules cleanly without writing messy if-else checks all over the code.

Another important part of the project was learning how to write defensive code. Creating a custom checked exception (VehicleNotAvailableException) showed me how real applications prevent mistakes like double-booking. Handling this exception in the user interface using try-catch blocks ensured that error messages are shown gently in pop-up boxes without crashing the entire application.

Working with multithreading was one of the most interesting parts. Implementing a Producer-Consumer queue with synchronized methods, wait(), and notify() helped me understand how background threads operate. It showed me how separating heavy tasks from the user interface thread keeps the desktop window smooth and responsive.

I also gained practical experience with SQLite and JDBC. Using PreparedStatements helped me understand why secure database queries are essential for protecting against errors and SQL injection. Combining this with Java Collections like HashMap and ArrayList provided fast in-memory access while keeping all data saved permanently on disk.

Overall, this project improved my coding skills, logical thinking, and confidence in Java development. It gave me a clear understanding of how desktop software is designed and showed how Java can be used effectively to solve real-world business problems.

REFERENCES

1. Oracle Java Standard Edition Documentation (JDK 17)
https://docs.oracle.com/en/java/javase/17/

2. SQLite JDBC Driver Documentation (Xerial Project)
https://github.com/xerial/sqlite-jdbc

3. SRM University AP — CSE 307 Course Curriculum
Department of Computer Science and Engineering, Advanced Object-Oriented Programming

4. Herbert Schildt, 'Java: The Complete Reference'
McGraw-Hill Education, 12th Edition (Reference for Java Swing, AWT, and Collections)

5. Joshua Bloch, 'Effective Java'
Addison-Wesley Professional, 3rd Edition (Guidelines on Inheritance, Exceptions, and Concurrency)

6. SQLite Official Documentation
https://www.sqlite.org/docs.html (ACID transaction handling and SQL syntax)